GOOGLE | 1POINT3ACRES | TOPIC 6 OF 10 | COMMON TOPIC

Binary Search and Sorting

4 questions to study here, each on its own page, with the full interview answer and code

5 more fit this topic but are covered in another topic: they are listed as SKIP at the end of the contents

4
MEDIUM

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Binary Search and Sorting | 4 questions | tap a title to jump to it

M	1. Cutting Ribbons (LC 1891)	2026-08-16
M	2. Progressive Array Partitioning	2026-08-11
M	3. Streaming Points: Emit K Within Distance	2026-06-23
M	4. First Bad Version with Parallel Search Follow-up	2026-06-15

SKIP: these fit this topic but have their pages in another topic (5)

M	SKIP Bounded Knapsack Variant	topic 2: Dynamic Programming
M	SKIP Active Communication Top Users	topic 4: Heaps, Stacks and Queues
M	SKIP Batch and Streaming Log Deduplication	topic 4: Heaps, Stacks and Queues
M	SKIP Russian Doll Envelopes	topic 2: Dynamic Programming
H	SKIP Median of Two Sorted Arrays (LC 4) with a Follow-up Variant	topic 4: Heaps, Stacks and Queues

STUDY HERE: topic 6, Binary Search and Sorting

Also fits: Arrays, Hashing and Two Pointers, Greedy and Math (listed there as SKIP, no pages). Site tags: array, binary-search, complexity-analysis, medium

MEDIUM | 1Point3Acres | onsite-coding | 1 of 4

Cutting Ribbons (LC 1891)

ASKED 2026-08-16 | 65 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 19.6 | 30 DAYS 0 | PRIORITY 147/228

ABOUT THIS QUESTION

Topic: Binary Search and Sorting

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1891 Cutting Ribbons

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/cutting-ribbons-max-equal-length

QUESTION

LC 1891 Cutting Ribbons: given wood lengths and K, maximize the equal piece length that yields at least K pieces. Follow-ups cover the search bound, floating-point lengths, and why complexity depends on the value range rather than the number of elements.SWEarraybinary-searchcomplexity-analysismedium FrequencySingle reportLast asked2026-08-16Stageonsite-coding

APPROACH

Implement the integer version from memory and prove the monotonic feasibility predicate.
Rework it with floating-point lengths and explain termination under both fixed-iteration and epsilon-based policies.
Practice stating the bound and complexity in terms of the numeric value domain without conflating it with input size.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether piece lengths must be integers, what to return if even length 1 fails (0), and the sizes of the array and k.

BRUTE FORCE TO SAY OUT LOUD

Try every length from the maximum down to 1 and count pieces. O(n * max).

OPTIMAL IDEA

Feasibility is monotone: if you can cut k pieces of length L you can for any smaller length. Binary search L over [1, max]: count = sum(r // L), move right if count >= k. O(n log max). A tighter upper bound is min(max, sum // k).

DRY-RUN THESE CASES

k larger than the total length (0), one ribbon, all equal ribbons, k = 1 (answer is the max ribbon).

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Real-valued lengths (fixed number of halvings, not an epsilon loop), why complexity depends on the value range, or an unlimited number of cuts.

PITFALLS

Overflow when summing. Searching from 0 and dividing by zero.

HOW TO THINK ABOUT IT

'Largest length such that at least K pieces fit' is a maximisation with a yes/no check. Larger lengths yield fewer pieces, so the check is monotone and you can binary search the answer. The check itself is one pass: sum of ribbon // length.

Basis: LeetCode 1891, Cutting Ribbons.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(ribbons, k):
    for L in range(max(ribbons), 0, -1):
        pieces = sum(r // L for r in ribbons)
        if pieces >= k:
            return L
    return 0
```

TIME	SPACE
O(n * max)	O(1)

WHY IT IS SLOW

Tries up to max(ribbon) lengths, each costing O(n); with ribbons up to 10^9 that is a billion probes. The check is monotone, so binary search needs only log2(max) probes.

OPTIMAL SOLUTION · Binary search the answer (integer lengths)

TIME	SPACE
O(n log(max length))	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why does the running time depend on the value range and not just on the number of ribbons?

Each probe costs O(n), and the number of probes is log2(max length) for the integer version. A huge length like 10^9 needs about 30 probes while 10 needs 4, even with the same n. For real numbers the probe count is set by the precision you promise, e.g. log2(max / epsilon).

FOLLOW-UP 2 What if the candidate length is given to you?

Then it is just the feasibility check: sum(r // L for r in ribbons) >= k, in O(n). The binary search exists only to find the best L; the check is the same function you reuse inside it.

FOLLOW-UP 3 What is a good upper bound for the search?

min(max(ribbons), sum(ribbons) // k). If the total length divided by k is smaller than the longest ribbon, no piece longer than that can ever give k pieces, which shrinks the range a little and avoids wasted probes. **Cost:** each probe is O(n); the tighter bound removes only a few of the log2(max) probes, so the total stays O(n log max).

FOLLOW-UP 4 Give the search a tighter upper bound.

No piece can be longer than the longest ribbon, and K pieces of length L need K * L total length, so L <= sum // K. Starting hi at min(max(ribbons), sum // K) saves a few probes and also returns 0 immediately when sum < K.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Cutting Ribbons (LC 1891)
#
# LC 1891 Cutting Ribbons: given wood lengths and K,
# maximize the equal piece length that yields at least K
# pieces. Follow-ups cover the search bound, floating-
# point lengths, and why complexity depends on the value
# range rather than the number of elements.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why does the running time depend on the value range
# and not just on the number of ribbons?
# 2. What if the candidate length is given to you?
# 3. What is a good upper bound for the search?
# 4. Give the search a tighter upper bound.
#
# INTERVIEW NOTES
# 'Largest length such that at least K pieces fit' is a
# maximisation with a yes/no check. Larger lengths yield
# fewer pieces, so the check is monotone and you can
# binary search the answer. The check itself is one pass:
# sum of ribbon // length.
#
# BRUTE FORCE (say it first): Try every length from the
# longest ribbon down
# For each length L from the maximum ribbon down to 1
# count the pieces and return the first L that gives at
# least K. Correct, but with ribbons up to 10^9 it would
# try a billion lengths. Time O(n * max). Space O(1).


# BRUTE FORCE (say it first): Try every length from the
# longest ribbon down | time O(n * max)
def max_length_brute(ribbons, k):
    for L in range(max(ribbons), 0, -1):
        if sum(r // L for r in ribbons) >= k:
            return L
    return 0
```

```
# HELPERS for the base question
def pieces(ribbons, length):
    # how many pieces of this length can we cut?
    return sum(r // length for r in ribbons)
```

```
# FOLLOW-UP 4: Give the search a tighter upper bound.
# Answer: No piece can be longer than the longest ribbon,
# and K pieces of length L need K * L total length, so L
# <= sum // K. Starting hi at min(max(ribbons), sum // K)
# saves a few probes and also returns 0 immediately when
# sum < K.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def upper_bound(ribbons, k):
    # no piece is longer than the longest ribbon, and
    # k pieces of length L need k * L total length
    return min(max(ribbons), sum(ribbons) // k)


# BASE QUESTION (optimal): Binary search the answer
# (integer lengths)
# Idea: If you can cut k pieces of length L you can cut k
# pieces of any shorter length, so feasibility is
# monotone.
def max_length(ribbons, k):
    # the best whole-number length (0 if impossible)
    lo, hi, best = 1, upper_bound(ribbons, k), 0
    while lo <= hi:
        mid = (lo + hi) // 2
        if pieces(ribbons, mid) >= k:
            # mid works: remember it and try longer
            best, lo = mid, mid + 1
        else:
            hi = mid - 1
    return best


# VARIANT: fractional lengths
def max_length_float(ribbons, k, rounds=50):
    # lengths may be fractions: fixed rounds of halving
    lo, hi = 0.0, float(max(ribbons))
    for _ in range(rounds):
        mid = (lo + hi) / 2
        # enough pieces: the answer is at least mid
        if sum(int(r // mid) for r in ribbons) >= k:
            lo = mid
        else:
            hi = mid
    return lo
```

```
if __name__ == "__main__":
    assert max_length([9, 7, 5], 3) == 5
    assert max_length([7, 5, 9], 4) == 4
    # not enough total length: no pieces at all
    assert max_length([1, 2], 10) == 0
    # the check alone, when the length is given
    assert pieces([9, 7, 5], 5) == 3
    assert upper_bound([9, 7, 5], 3) == 7
    # fractional lengths: 3 pieces from one ribbon of 1
    assert abs(max_length_float([1], 3) - 1 / 3) < 1e-9
    assert max_length_float([1], 0 + 2) > 0.49
```

NOTES FROM CANDIDATES

The interviewer allowed the candidate to abandon an initial greedy/sorting direction, then unlocked the problem with one question: what changes if a candidate length is supplied?
Expect the follow-ups, not the base implementation, to consume most of the round.
For the integer version, initialize the best answer to 0, search only positive candidate lengths, and return 0 when even length 1 cannot produce K pieces. The feasibility predicate is monotone: once a length is infeasible, every larger length is also infeasible.

PREPARATION

Implement the integer version from memory and prove the monotonic feasibility predicate.
Rework it with floating-point lengths and explain termination under both fixed-iteration and epsilon-based policies.
Practice stating the bound and complexity in terms of the numeric value domain without conflating it with input size.

STUDY HERE: topic 6, Binary Search and Sorting

Also fits: Arrays, Hashing and Two Pointers (listed there as SKIP, no pages). Site tags: array, hashmap, sorting, medium

MEDIUM | 1Point3Acres | phone-screen | 2 of 4

Progressive Array Partitioning

ASKED 2026-08-11 | 70 DAYS AGO | LAST 90 DAYS

STUDY: TALK | **NO LEETCODE FREQUENCY** | **PRIORITY 198/228**

ABOUT THIS QUESTION

Topic: Binary Search and Sorting

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-08-11 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/array-partition-unique-frequencies-straights

QUESTION

EXAMPLES

Stage 1

```
[1, 2, 3, 4, 2, 3, 3, 4, 4, 4] -> true because the
    four value-groups have different sizes.
[1, 2, 3] -> false because every value-group has size
    one.
```

Stage 2

```
[1, 2, 2, 3, 3, 4, 4, 5, 5, 6] -> true.
[4, 5, 6, 7, 7, 8, 9] -> false.
```

APPROACH

Implement both stages against repeated values, singletons, empty input, and inputs with many duplicate counts.
Practice stating the partition invariant and asking about run length, duplicate consumption, and whether every element must be used exactly once before writing code.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what the groups are and the rule for each stage. From the examples, the first stage looks like: true when the counts of all distinct values are different from each other (1:1, 2:2, 3:3, 4:4 is true; all counts 1 is false). The second stage is not clear from the examples alone, so ask for its exact rule and test it on both examples before coding.

BRUTE FORCE TO SAY OUT LOUD

Try every way to group the elements and test the rule. Exponential.

OPTIMAL IDEA

For stage one: count each value with a hash map, then check that the multiset of counts has no repeats (a set of counts has the same size as the number of values). O(n). Keep the data model (a counter) easy to adapt, because the second stage arrives only after the first is written.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

The four examples in the statement, an empty array, a single value, many values with equal counts.

LIKELY FOLLOW-UPS

Stage two, which changes the grouping rule; keep helper functions small so it is a local change.

PITFALLS

Coding stage two from a guess. Not reconciling the rule with both examples.

HOW TO THINK ABOUT IT

The statement arrives in stages, so keep the data model easy to adapt: count the values first, because any rule about 'groups of equal values' is a rule about the counts. Stage 1 is then 'are all counts different', which is a set-size comparison. Before stage 2, reconcile the new rule with every example; here a natural guess (LeetCode 659) contradicts the second example, which is exactly the situation where you ask.

Basis: Written. Stage 1 is inferred from the examples: true when every distinct value occurs a different number of times. The Stage 2 rule is NOT recoverable from the two examples, so only a candidate reading is shown; ask the interviewer for the exact rule and test it against both examples before coding.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    counts = Counter(a)
    for x, y in combinations(counts, 2):
        if counts[x] == counts[y]:
            return False
    return True
```

TIME	SPACE
O(d^2) where d is the number of distinct values	**O(d)**

WHY IT IS SLOW

O(d^2) pair checks for d distinct values. Comparing the number of distinct counts with the number of distinct values (a set) is O(n).

OPTIMAL SOLUTION · Counter, then compare the set of counts with the number of values

TIME	SPACE
O(n)	**O(d)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Stage 2 arrives. What do you do?

Do not guess. Write down what the second example pair implies, and check any candidate rule against BOTH. The common reading 'split into consecutive runs of length at least 3' gives true for [4, 5, 6, 7, 7, 8, 9], but the statement says false, so there is an extra condition (such as run lengths or a count rule) to ask about.

FOLLOW-UP 2 Keep the code adaptable between stages.

Put the counting in one helper and each stage's rule in its own small function that takes the counts. Stage 2 then replaces a function instead of rewriting the pipeline.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Progressive Array Partitioning
#
# Examples: Stage 1 [1, 2, 3, 4, 2, 3, 3, 4, 4, 4] -> true
# because the four value-groups have different sizes. [1,
# 2, 3] -> false because every value-group has size one.
# Stage 2 [1, 2, 2, 3, 3, 4, 4, 5, 5, 6] -> true. [4, 5, 6
,
# 7, 7, 8, 9] -> false.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Stage 2 arrives. What do you do?
# 2. Keep the code adaptable between stages.
#
# INTERVIEW NOTES
# The statement arrives in stages, so keep the data model
# easy to adapt: count the values first, because any rule
# about 'groups of equal values' is a rule about the
# counts. Stage 1 is then 'are all counts different',
# which is a set-size comparison. Before stage 2,
# reconcile the new rule with every example; here a
# natural guess (LeetCode 659) contradicts the second
# example, which is exactly the situation where you ask.
#
# BRUTE FORCE (say it first): Compare every pair of counts
# Count each value, then check every pair of counts for
# equality. Fine for small d and a clear statement of the
# rule. Time O(d^2) where d is the number of distinct
# values. Space O(d).


from collections import Counter


# BRUTE FORCE (say it first): Compare every pair of counts
# | time O(d^2) where d is the number of distinct values
def distinct_counts_brute(nums):
    counts = list(Counter(nums).values())
    return all(
        counts[i] != counts[j]
        for i in range(len(counts))
        for j in range(i + 1, len(counts))
    )


# BASE QUESTION (optimal): Counter, then compare the set
# of counts with the number of values
```

```
# Idea: All counts are different exactly when the set of
# counts has as many elements as there are distinct
# values. One pass to count, one pass over the counts.
def stage1(counts):
    # true when every value occurs a different number of
    # times: the counts have no repeats
    return len(set(counts.values())) == len(counts)


# FOLLOW-UP 1: Stage 2 arrives. What do you do?
# Answer: Do not guess. Write down what the second example
# pair implies, and check any candidate rule against BOTH.
def stage2_candidate(nums, min_len=3):
    # ONE reading of stage 2 (LeetCode 659): split the
    # array into runs of consecutive values, each at
    # least min_len long. Check it against BOTH examples.
    left, ends = Counter(nums), Counter()
    for x in nums:
        if left[x] == 0:
            continue
        left[x] -= 1
        if ends[x - 1] > 0:
            # extend a run that ended at x - 1
            ends[x - 1] -= 1
            ends[x] += 1
        elif left[x + 1] > 0 and left[x + 2] > 0:
            # start a new run x, x + 1, x + 2
            left[x + 1] -= 1
            left[x + 2] -= 1
            ends[x + 2] += 1
        else:
            return False
    return True


# HELPERS for the checks below
def counts_of(nums):
    # every rule about groups of equal values is a
    # rule about these counts, so compute them once
    return Counter(nums)


# each stage is one small function, so a new rule
# replaces one entry instead of the whole pipeline
RULES = {1: lambda nums: stage1(counts_of(nums)),
```

```
         2: stage2_candidate}


def check(stage, nums):
    return RULES[stage](nums)


if __name__ == "__main__":
    # stage 1 examples
    assert check(1, [1, 2, 3, 4, 2, 3, 3, 4, 4, 4])
    assert not check(1, [1, 2, 3])
    # stage 2 example that the candidate gets right
    assert check(2, [1, 2, 2, 3, 3, 4, 4, 5, 5, 6])
    # the second stage-2 example says False, but this
    # reading says True: the rule is incomplete, so ask
    assert check(2, [4, 5, 6, 7, 7, 8, 9]) is True
```

NOTES FROM CANDIDATES

The interviewer presents Stage 2 only after the first implementation, so keep the initial data model easy to adapt.
Before coding the straight variant, reconcile the length rule with both examples; otherwise a superficially valid decomposition can contradict the expected result.
Evaluation emphasized the efficiency of the chosen data structures, not only whether the sample cases passed.
Solution skeleton
Stage 1: count each distinct value, then verify that the frequency values are themselves unique. A hash map plus a set gives O(n) time and O(u) space for u distinct values.
Stage 2: first pin down a group size k. For the fixed-size interpretation, reject when n % k != 0, process keys in ascending order, and whenever the smallest remaining key x has multiplicity c, remove c copies from every key in [x, x + k). Any missing count makes the partition impossible. Sorting the u keys gives O(n + u log u) time and O(u) space.
Do not let length-one groups make every input trivially valid unless the interviewer explicitly allows them; the negative example requires a stronger group-size contract.

PREPARATION

Implement both stages against repeated values, singletons, empty input, and inputs with many duplicate counts.
Practice stating the partition invariant and asking about run length, duplicate consumption, and whether every element must be used exactly once before writing code.

STUDY HERE: topic 6, Binary Search and Sorting

Also fits: Arrays, Hashing and Two Pointers, Design, Streaming and Scale (listed there as SKIP, no pages). Site tags: data-structure, sorting, sliding-window, medium, ambiguous-spec

MEDIUM | 1Point3Acres | onsite-coding | 3 of 4

Streaming Points: Emit K Within Distance

ASKED 2026-06-23 | 119 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 199/228**

ABOUT THIS QUESTION

Topic: Binary Search and Sorting

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-06-23 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/streaming-points-k-within-distance

QUESTION

Process a stream of float positions. Whenever three points satisfy the distance condition, output and remove those three points; a follow-up generalizes from 3 points to K points. The important ambiguity is whether the condition means a span of length **d** or pairwise distance within **d**.SWEdata-structuresorting sliding-windowmediumambiguous-specFrequencySingle reportLast asked2026-06-23Stageonsite-coding

APPROACH

Practice range queries over sorted points with insertion and deletion.
Write the K-generalization in words before coding: identify the local candidate window, test the predicate, then remove exactly the emitted points.
Rehearse explaining float comparison, tie-breaking, and what to do when multiple groups qualify.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the condition means all K points fit in a span of length d or every pair is within d (on a line these are equivalent), whether to emit as soon as a group exists, and how to choose among overlapping groups.

BRUTE FORCE TO SAY OUT LOUD

Keep a list, and after each point sort it and scan for a group. O(n log n) per point.

OPTIMAL IDEA

Keep points in a sorted list (bisect.insort). After inserting x only windows containing x can be new, so look at the K-wide windows of sorted neighbors around its index; if the largest minus the smallest in a window is at most d, emit and remove those K points. O(log n) to locate plus the window size.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

Exactly d apart, duplicates, points arriving out of order, two overlapping candidate groups, K = 1.

LIKELY FOLLOW-UPS

General K (the same code), 2D points, or a time-based expiration.

PITFALLS

Using epsilon-free float comparison blindly. Checking only the K points directly before x.

HOW TO THINK ABOUT IT

On a line, K points are all within distance d exactly when the largest minus the smallest is at most d, so only K consecutive points in sorted order can form a group. Keep the points sorted and look only where something changed: a window that contains the new point, or the seam left by removed points.

Basis: Written. Assumed rules (confirm them): after each new point, while some K consecutive points (in sorted order) span at most d, emit the lowest such group and remove it; on a line 'all within d' and 'span at most d' are the same condition.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
points = []
def on_point(x):
    points.append(x)
    points.sort()
    emitted = []
    while True:
        group = find_group()
        if group is None:
            return emitted
        emitted.append(group)
        remove(points, group)
def find_group():
    for i in range(len(points) - k + 1):
        if points[i + k - 1] - points[i] <= d:
            return points[i:i + k]
    return None
```

TIME	SPACE
O(n log n) per point	O(n)

WHY IT IS SLOW

Every arrival re-sorts and rescans the whole list, O(n log n). A sorted list with binary search checks only the windows around the new point.

OPTIMAL SOLUTION · Sorted list, check only windows touching the change

TIME	SPACE
O(log n + K) per check, plus list shifting	O(n)

OPTIMAL SOLUTION · Sorted list, check only windows touching the change

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if 'within d' means every pair, and points are in 2D?

On a line pairwise distance equals the span, so nothing changes. In 2D it becomes a clique problem: candidates must be neighbours in a spatial grid of cell size d, and you test the K-subset of nearby points; that is much harder, so ask whether the 1D reading is intended.

FOLLOW-UP 2 Floating-point coordinates.

Compare with the exact subtraction hi - lo <= d (not by building a range around the target), and agree on an epsilon policy only if the interviewer wants one. Do not compare floats for equality.

FOLLOW-UP 3 Several groups qualify at once.

Agree on a rule (lowest start, as coded) so output is deterministic, and note that emitting one group can create or destroy others.

FOLLOW-UP 4 The condition is 'every pair within d', not 'span at most d'.

On a line they are the same: the largest pairwise distance is max - min, so checking the span is enough (verified against checking all pairs). In 2D they differ and become a clique problem.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Streaming Points: Emit K Within Distance
#
# Process a stream of float positions. Whenever three
# points satisfy the distance condition, output and remove
# those three points; a follow-up generalizes from 3
# points to K points. The important ambiguity is whether
# the condition means a span of length d or pairwise
# distance within d.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if 'within d' means every pair, and points are
# in 2D?
# 2. Floating-point coordinates.
# 3. Several groups qualify at once.
# 4. The condition is 'every pair within d', not 'span at
# most d'.
#
# INTERVIEW NOTES
# On a line, K points are all within distance d exactly
# when the largest minus the smallest is at most d, so
# only K consecutive points in sorted order can form a
# group. Keep the points sorted and look only where
# something changed: a window that contains the new point,
# or the seam left by removed points.
#
# BRUTE FORCE (say it first): Re-sort and scan every
# window after each point
# Sort all points, slide a window of K, and emit the
# lowest group that fits, repeating until none does.
# Obviously correct; the whole list is re-examined for
# every new point. Time O(n log n) per point. Space O(n).


from bisect import bisect_left, insort


# BRUTE FORCE (say it first): Re-sort and scan every
# window after each point | time O(n log n) per point
class PointStreamBrute:
    def __init__(self, k, d):
        self.k, self.d, self.pts = k, d, []

    def add(self, x):
        self.pts.append(x)
        out = []
        while True:
            self.pts.sort()
            for i in range(len(self.pts) - self.k + 1):
                if (
                    self.pts[i + self.k - 1]
                    - self.pts[i]
                    <= self.d
                ):
                    out.append(self.pts[i : i + self.k])
                    del self.pts[i : i + self.k]
                    break
            else:
                return out


# BASE QUESTION (optimal): Sorted list, check only windows
# touching the change
# Idea: Insert with bisect. Because no window existed
# before, a new window must contain the new point, so test
# the K windows that include its position.
class PointStream:
    def __init__(self, k, d):
        # emit when k points fit in a span of d
        self.k, self.d = k, d
        # all points still waiting, kept sorted
        self.points = []

    def window_start(self, pos):
        # first window of k neighbours that includes pos
        lo = max(0, pos - self.k + 1)
        hi = min(pos, len(self.points) - self.k)
        for i in range(lo, hi + 1):
            last = self.points[i + self.k - 1]
            span = last - self.points[i]
            if span <= self.d:
                return i
        return None

    def add(self, x):
        # insert in order, then emit every window made by
        # x
        insort(self.points, x)
        out, pos = [], bisect_left(self.points, x)
        while True:
            i = self.window_start(pos)
```

```
            if i is None:
                return out
            # remove the group so it is emitted only once
            out.append(self.points[i:i + self.k])
            del self.points[i:i + self.k]
            pos = i


# FOLLOW-UP 4: The condition is 'every pair within d', not
# 'span at most d'.
# Answer: On a line they are the same: the largest
# pairwise distance is max - min, so checking the span is
# enough (verified against checking all pairs). In 2D they
# differ and become a clique problem.
def all_pairs_within(group, d):
    # "every pair within d": on a line this equals the
    # span
    pairs = [(a, b) for a in group for b in group]
    return all(abs(a - b) <= d for a, b in pairs)


if __name__ == "__main__":
    s = PointStream(3, 1.0)
    assert s.add(0.5) == []
    assert s.add(5.0) == []
    assert s.add(1.0) == []
    # 0.5, 1.0, 1.4 span 0.9 <= 1.0
    out = s.add(1.4)
    assert out == [[0.5, 1.0, 1.4]]
    # the far point is still waiting
    assert s.points == [5.0]
    # on a line, span and pairwise checks agree
    for group in out:
        assert all_pairs_within(group, 1.0)
        assert max(group) - min(group) <= 1.0
    # k = 2 groups closest neighbours as they appear
    t = PointStream(2, 0.5)
    assert t.add(1.0) == []
    assert t.add(1.4) == [[1.0, 1.4]]
```

NOTES FROM CANDIDATES

A balanced ordered set / TreeSet-style structure is a natural fit because each new point only needs nearby neighbors.
Be careful with floating-point comparison and epsilon policy.
A wrong clarification can lead to checking subset(target, target + d) when the interviewer expects a wider candidate range around the target. Confirm the predicate with a tiny numeric example before coding.

PREPARATION

Practice range queries over sorted points with insertion and deletion.
Write the K-generalization in words before coding: identify the local candidate window, test the predicate, then remove exactly the emitted points.
Rehearse explaining float comparison, tie-breaking, and what to do when multiple groups qualify.

STUDY HERE: topic 6, Binary Search and Sorting

Also fits: Design, Streaming and Scale (listed there as SKIP, no pages). Site tags: binary-search, parallelism, concurrency, medium

MEDIUM | 1Point3Acres | phone-screen | 4 of 4

★ TOP 75: rank 72 in the Medium and Hard priority list

★ First Bad Version with Parallel Search Follow-up

ASKED 2026-06-15 | 127 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE **ALL TIME 41.1** **30 DAYS 33** **PRIORITY 47/228**

ABOUT THIS QUESTION

Topic: Binary Search and Sorting

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #278 First Bad Version

LeetCode match: LC 278 First Bad Version

Frequency: Single report | Last asked: 2026-06-15 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/first-bad-version-parallel-search

QUESTION

Phone-screen R1 coding round. The base problem is the standard first-bad-version monotonic search — given an `isBadVersion(v)` oracle over versions `1..n`, find the first bad version with a single binary search. The graded follow-up requires parallelizing the search: partition the range into buckets and search them concurrently (bucket + binary search), with working code expected.SWEbinary-searchparallelismconcurrencymediumFrequencySingle reportLast asked2026-06-15Stagephone-screen

APPROACH

Write the single-threaded binary search first and verify it on small n.
Then implement the bucketed parallel version with real concurrency primitives (Python concurrent.futures.ThreadPoolExecutor, or Java ExecutorService / CompletableFuture): split [1, n] into k buckets, probe the k boundaries concurrently, pick the bucket where the predicate flips, and binary-search within it. Practice getting this to compile and run in under 20 minutes.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what the oracle is (isBadVersion is monotone), what concurrency primitive to use (ThreadPoolExecutor, Java's ExecutorService), and which metric matters: wall-clock time or total calls.

BRUTE FORCE TO SAY OUT LOUD

Check every version from 1. O(n) calls.

INTERVIEW-DAY PLAN

OPTIMAL IDEA

Base: binary search, O(log n) calls. Parallel: each round fires up to k probes concurrently at evenly spaced points, finds the first probe that is bad, and narrows the interval to the gap between it and the previous probe. Each round shrinks the interval by k + 1, so there are about log base (k + 1) of n sequential rounds. Use a thread pool and map the probes in one call.

DRY-RUN THESE CASES

n = 1, the first version bad, the last version bad, n smaller than the number of workers.

LIKELY FOLLOW-UPS

Rate limiting, retries when a probe fails, and caching results.

PITFALLS

Mutating shared state from threads. Assuming more threads always means fewer total calls (they increase it).

HOW TO THINK ABOUT IT

'First bad version' is a boundary in a sorted yes/no sequence, which is binary search. The parallel part is a different trade: instead of one probe per round, send several probes in the same round and keep the gap that contains the boundary. Fewer rounds, more total calls.

Basis: LeetCode 278, First Bad Version, plus a concurrent k-ary search follow-up.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n):
    for v in range(1, n + 1):
        if is_bad_version(v):
            return v
```

TIME	SPACE
O(n)	O(1)
calls	

WHY IT IS SLOW

One call per version, O(n) calls; with a billion versions and a slow check that is far too many. The check is monotone, so binary search needs only log2(n) calls, and probing several points in parallel needs fewer rounds.

OPTIMAL SOLUTION · Base problem: binary search

TIME	SPACE
O(log n)	O(1)
calls	

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Parallel: probe k points at once, then recurse into the bucket that flips

TIME	SPACE
O(log_{k+1} n) rounds	O(k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What does parallelism buy you, and what does it cost?

It cuts the number of sequential rounds from log2(n) to log_{k+1}(n), so wall-clock time drops when each probe has high latency. It increases total probes (about k per round instead of 1), so it costs more calls and money. Say which metric you are optimizing before choosing k.

FOLLOW-UP 2 What if the oracle is rate limited or can fail?

Cap the pool size to the rate limit, retry transient failures with backoff (probes are idempotent), and never advance the interval from a probe that errored. Cache results by version so a retry or a later round never repeats a call.

FOLLOW-UP 3 Count how many rounds each approach needs.

Single binary search: log2(n) rounds of one probe each. k workers: about log base (k + 1) of n rounds with k probes each. For n = 10^9, one probe per round needs 30 rounds, while 7 probes per round need 10.

FOLLOW-UP 4 Show how many rounds and calls the parallel search uses.

With k workers each round shrinks the interval by about k + 1, so the number of sequential rounds is about log base (k + 1) of n and the total calls at most k times that. For n = 10^6 and 7 workers that is about 7 rounds and 49 calls, against 20 rounds and 20 calls for the sequential search: fewer rounds, more calls.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: First Bad Version with Parallel Search
# Follow-up
#
# Phone-screen R1 coding round. The base problem is the
# standard first-bad-version monotonic search — given an
# isBadVersion(v) oracle over versions 1..n, find the
# first bad version with a single binary search. The
# graded follow-up requires parallelizing the search:
# partition the range into buckets and search them
# concurrently (bucket + binary search), with working code
# expected.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What does parallelism buy you, and what does it cost?
# 2. What if the oracle is rate limited or can fail?
# 3. Count how many rounds each approach needs.
# 4. Show how many rounds and calls the parallel search
# uses.
#
# INTERVIEW NOTES
# 'First bad version' is a boundary in a sorted yes/no
# sequence, which is binary search. The parallel part is a
# different trade: instead of one probe per round, send
# several probes in the same round and keep the gap that
# contains the boundary. Fewer rounds, more total calls.
#
# BRUTE FORCE (say it first): Check versions one by one
# Call isBadVersion for 1, 2, 3, ... and stop at the first
# bad one. Obviously correct and the oracle; with a
# billion versions and a slow check it is far too many
# calls. Time O(n) calls. Space O(1).


from concurrent.futures import ThreadPoolExecutor
from threading import Lock


# BRUTE FORCE (say it first): Check versions one by one |
# time O(n) calls
def first_bad_linear(n, is_bad):
    return next(v for v in range(1, n + 1) if is_bad(v))


# BASE QUESTION (optimal): problem: binary search
# Idea: isBad is monotone (all good, then all bad), so
# keep the invariant 'the first bad version is in [lo,
# hi]' and halve the interval. Use lo + (hi - lo) // 2 in
# languages with fixed-width integers.
def first_bad_version(n, is_bad):
    # the base question: binary search on versions 1..n
    lo, hi = 1, n
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if is_bad(mid):
            # the first bad version is mid or earlier
            hi = mid
        else:
            lo = mid + 1
    return lo


# PARALLEL SEARCH helper
def probe_points(lo, hi, workers):
    # evenly spaced versions inside [lo, hi - 1]
    step = max(1, (hi - lo + 1) // (workers + 1))
    pts = {min(hi - 1, lo + step * (i + 1) - 1)
           for i in range(workers)}
    return sorted(pts)


# FOLLOW-UP 1: What does parallelism buy you, and what
# does it cost?
# Answer: It cuts the number of sequential rounds from
# log2(n) to log_{k+1}(n), so wall-clock time drops when
# each probe has high latency. It increases total probes
# (about k per round instead of 1), so it costs more calls
# and money.
# FOLLOW-UP 3: Count how many rounds each approach needs.
# Answer: Single binary search: log2(n) rounds of one
# probe each. k workers: about log base (k + 1) of n
# rounds with k probes each. For n = 10^9, one probe per
```

```
# round needs 30 rounds, while 7 probes per round need 10.
# FOLLOW-UP 4: Show how many rounds and calls the parallel
# search uses.
# Answer: With k workers each round shrinks the interval
# by about k + 1, so the number of sequential rounds is
# about log base (k + 1) of n and the total calls at most
# k times that.
def first_bad_parallel(n, is_bad, workers=4):
    # invariant: the answer is in [lo, hi]
```

```
    lo, hi, cache, rounds = 1, n, {}, 0
    with ThreadPoolExecutor(workers) as pool:
        while lo < hi:
            points = probe_points(lo, hi, workers)
            # ask the oracle about new points at the same
            # time
            fresh = [p for p in points if p not in cache]
            answers = pool.map(is_bad, fresh)
            for p, bad in zip(fresh, answers):
                cache[p] = bad
            rounds += 1
            bad_points = [p for p in points if cache[p]]
            if not bad_points:
                # every probe was good: answer is after
                # them
                lo = points[-1] + 1
            else:
                first = bad_points[0]
                # the answer sits between the last good
                # probe before it and the first bad one
                before = [p for p in points if p < first]
                if before:
                    lo = before[-1] + 1
                hi = first
    return lo, rounds, len(cache)


if __name__ == "__main__":
    calls, lock = [], Lock()

    def oracle(first_bad):
        def is_bad(v):
            with lock:
                calls.append(v)
            return v >= first_bad
        return is_bad

    cases = ((1, 1), (10, 1), (10, 10), (1000, 637))
    for n, bad in cases:
        assert first_bad_version(n, oracle(bad)) == bad
        ans, rounds, used = first_bad_parallel(
            n, oracle(bad), workers=4)
        assert ans == bad
    # fewer rounds than the 20 of a plain binary search
    ans, rounds, used = first_bad_parallel(
```

```
        10 ** 6, oracle(123456), workers=7)
    assert ans == 123456 and rounds < 20
    assert used <= 7 * rounds
```

NOTES FROM CANDIDATES

The base is trivial; budget your time for the parallel follow-up and aim to get a complete, compiling version through even if it has minor bugs — finishing the parallel version is what passes this round.

Be explicit about what the parallel design optimizes: wall-clock latency (fire many isBadVersion probes at once) versus total number of calls. Splitting into k buckets and probing each boundary concurrently localizes the transition in one parallel round, then a binary search finishes it.

Common bugs: off-by-one when mapping a bucket index back to an absolute version range; the transition sitting exactly on a bucket boundary; integer overflow in mid if you write (lo + hi) / 2 instead of lo + (hi - lo) // 2.

PREPARATION

Write the single-threaded binary search first and verify it on small n.

Then implement the bucketed parallel version with real concurrency primitives (Python concurrent.futures.ThreadPoolExecutor, or Java ExecutorService / CompletableFuture): split [1, n] into k buckets, probe the k boundaries concurrently, pick the bucket where the predicate flips, and binary-search within it. Practice getting this to compile and run in under 20 minutes.